

VR-Forms. Preprint.

Part II. The Formal Language

Vitaly Reznik

2026

II.0. Introduction

Part I set out the general position: the formal register as speech without ontological commitments. The present part translates this position into formal apparatus.

Contents of Part II: precise definition of a formal term; syntax of the formal register; rules of formation of formal terms; the relation of a formal term to an operational set of VR-Sets.

Part II is technically minimal. Its task is to fix the apparatus on which Parts III–VII will build substantive results. No new ontological moves are made here; everything rests on Part I.

II.1. Alphabet and terms

The formal language of VR-Forms is built over the same alphabet as VR-Sets, extended by a register-separator.

Alphabet

The alphabet of the formal language contains:

- the symbol $\emptyset$;
- the functional symbol t (successor);
- logical connectives $\rightarrow, \neg, \wedge, \vee, \leftrightarrow$;
- quantifiers $\forall, \exists$;
- the membership relation $\in$;
- the operational identity relation $\equiv$ (from VR-Sets);
- variables $x, y, z, \dots$;
- brackets $\{, \}, (,)$;
- the register-separator symbol $\ulcorner \cdot \urcorner$ (see below).

The alphabet remains finite, so that countability of the syntax is preserved (VR-Sets §VI.1).

Terms and formulas

Terms and formulas are defined inductively in the standard way:

- $\emptyset$ is a term;
- if τ is a term, then $t(\tau)$ is a term;
- variables are terms;

— a description of the form $\{x : \varphi(x)\}$, where φ is a formula with one free variable x , is a term;

— other formulas are built from atomic $x \in y$, $x \equiv y$ by connectives and quantifiers.

So far this is exactly the language of VR-Sets. The difference with VR-Forms arises in the rules of **use** of terms, not in their syntax.

II.2. Register-separator

The same term may figure in the ontological or in the formal register. To make the distinction explicit in writing, a syntactic separator is introduced.

Definition II.1 (register-separator)

For any term τ , the notation $\ulcorner \tau \urcorner$ means: the term τ is considered in the formal register. The notation τ without brackets means: the term is considered in the ontological register.

$\ulcorner \cdot \urcorner$ is not a function or operation. It is a sign of the **mode of consideration**. It indicates in which of the two registers a given term is read.

Examples

— $\emptyset$ is the empty set in the ontological register. By Lemma 2 of VR-Sets it is the unique operational object with empty functionality.

— $\ulcorner \emptyset \urcorner$ is a formal term, the description «of the empty». In the formal register this is simply a formula; no ontological commitments. (In practice for $\emptyset$ the ontological and formal registers coincide, since the description directly corresponds to an operational object; the separator is here redundant but admissible.)

— $\ulcorner \{x : x \notin x\} \urcorner$ is the Russell formal term. Admissible in the formal register. The corresponding ontological term $\{x : x \notin x\}$ does not specify a set (VR-Sets §II.3).

— $\ulcorner \mathbb{R} \urcorner$ is the formal term «classical set of reals». Admissible in the formal register.

— $\mathbb{R}_{\text{VR}}$ is the notation for the operational set of computable reals of VR-Sets. In the ontological register. Not the same as $\ulcorner \mathbb{R} \urcorner$.

A note on practice of notation

In the work, the separator $\ulcorner \cdot \urcorner$ is used explicitly where the register is otherwise ambiguous. In contexts where the register is clear from the text (for instance, «classical $\mathbb{R}$ » — formal register; « $\mathbb{R}_{\text{VR}}$ » — ontological), the separator is omitted.

II.3. Definition of a formal term

We now give the central definition of the present work.

Definition II.2 (formal term)

A **formal term** is any correctly built syntactic record of a description, considered in the formal register.

Formally: a formal term is a pair (τ, F) , where τ is a syntactically correct term of the language of VR-Forms, and F is the register-marker indicating «formal». The notation $\ulcorner \tau \urcorner$ denotes this pair.

Commentary

(1) A formal term is a **pair**: syntax plus indication of mode. The same τ specifies different entities depending on the register: an operational set (in the ontological) or a formal term (in the formal).

(2) «Correctly built syntactic record» means: the term is built according to the rules of §II.1. No other conditions — operability, consistency, constructivity — are imposed. A formal term may be contradictory, non-operational, refer to uncountable collections, describe the impossible. All this is permitted.

(3) In the formal register there is no distinction between «existing» and «non-existing» formal terms. All formal terms are formal terms. The differences in their content (countable, paradoxical, non-mathematical) are properties of **the description**, not properties of being.

II.4. The principle of forms

Corresponding to the closure principle of VR-Sets (every describable operational functionality is a set), there is in VR-Forms a parallel principle pertaining to the formal register.

The principle of forms

Every syntactically correct record of a description specifies a formal term.

No restrictions: operability is not required, consistency is not required, describability in any special sense is not required. It suffices that the record is syntactically correct by §II.1.

Comparison with the closure principle

The closure principle of VR-Sets applies in the ontological register and requires operational describability. Contradictory descriptions (Russell) do not specify sets. This is a **conditional** principle: given operability — there is a set.

The principle of forms applies in the formal register and requires nothing other than syntactic correctness. This is an **unconditional** principle: every description is a formal term.

Parallel and difference: both principles generate objects of their registers; but in the ontological register the objects «are» (as actions upon $\emptyset$), and in the formal — «are formulas». The difference is not in the statement of the principles, but in the ontological load of the objects they generate.

II.5. Relations and operations on formal terms

Formal terms are formulas. Yet one may work with formulas: formulate statements, derive consequences, compare. This work is work in the formal register.

Membership

The notation $\ulcorner x \in \tau \urcorner$ means: the formal statement « x belongs to the formal term τ ». The truth value of this statement is determined by the inference rules of the formal register, not by the ontology of VR-Sets.

Example: let $\tau = \ulcorner \{x : \varphi(x)\} \urcorner$. Then $\ulcorner y \in \tau \urcorner$ is true in the formal register if and only if $\varphi(y)$ is true (in the formal register). This is the standard scheme of comprehension for formal terms.

Comprehension in the formal register is **not restricted** by an axiom of separation. Unlike ZF, where comprehension applies only to elements of already existing sets, in the formal register any description $\{x : \varphi(x)\}$ directly specifies a formal term. This is what makes the Russell formal term a legitimate object of the formal register.

Equality and identity

In the formal register the equality of two formal terms $\ulcorner \tau_1 \urcorner = \ulcorner \tau_2 \urcorner$ is a statement about their **descriptions**. Two formal terms are equal if they are given by one and the same formula (up to renaming of bound variables) or if the equivalence is provable in the formal register.

This differs from $\equiv$ of VR-Sets — the operational identity requiring coincidence of functionalities. In the formal register there are no functionalities; there are formulas.

Logical operations

Connectives and quantifiers apply to formulas of the formal register in the usual way. Classical logic is preserved. No special inference rules for formal terms are introduced.

This is an important technical feature: the formal register is **not a new logic**; it is the same logical apparatus applied to a new type of objects (formal terms instead of operational sets).

II.6. Difference between a formal term and an operational set

Let us gather in one place a systematic comparison.

What is shared

- The same syntax: both formal terms and operational sets are written by formulas of one language.
- The same logic: classical, with the usual rules.
- The same starting primitive: $\emptyset$ is the base object of both registers.

What differs

Register. An operational set is in the ontological. A formal term is in the formal.

Ontological load. An operational set is (as action upon $\emptyset$). A formal term is a formula.

Condition of existence. An operational set requires operational describability (closure principle). A formal term requires only syntactic correctness (principle of forms).

Paradoxes. Contradictory descriptions do not specify operational sets. Contradictory descriptions specify formal terms without restriction.

Universe. All operational sets form the countable universe of VR-Sets. All formal terms form a countable syntactic universe (since formulas are finite sequences over a finite alphabet).

Identity. Operational identity $\equiv$ is the coinductive coincidence of functionalities. Formal identity $=$ is syntactic or formal-register-provable coincidence of descriptions.

Connection

To every operational set A there corresponds exactly one formal term $\ulcorner A \urcorner$ — the description of A itself, considered in the formal register. Conversely: not every formal term has an operational correlate. Those that do form a distinguished subfamily of formal terms — let us call it the **operationally realisable** formal terms.

This subfamily is countable (by countability of operational sets). The set of all formal terms is likewise countable (by finiteness of the alphabet). Both registers thus remain within the countable ontology of VR.

II.7. Operationally realisable formal terms

We introduce this notion precisely — it will be needed in Part IV for the definition of transit.

Definition II.3

A formal term $\ulcorner \tau \urcorner$ is called **operationally realisable** if in the ontological register there exists an operational set A such that the description τ corresponds to the functionality A .

Equivalently: $\ulcorner \tau \urcorner$ is operationally realisable if τ (without the formal-register marker) is an admissible description in VR-Sets satisfying the closure principle.

Examples of realisable formal terms

$\ulcorner \emptyset \urcorner, \ulcorner \{\emptyset\} \urcorner, \ulcorner \omega \urcorner, \ulcorner \emptyset_{VR(\omega)} \urcorner, \ulcorner \mathbb{R}_{VR} \urcorner$ — all of these are realisable. Each corresponds to an operational set of VR-Sets.

Any description of a computable functionality is an operationally realisable formal term.

Examples of non-realisable formal terms

$\ulcorner \{x : x \notin x\} \urcorner$ — the Russell term. Not realisable (the description is contradictory).

$\ulcorner \mathbb{R} \urcorner$ (classical) — not realisable (uncountable, not operationally describable).

$\ulcorner \text{the Vitali set} \urcorner$ — not realisable (requires choice over an uncountable family).

$\ulcorner \text{dragon} \urcorner$ — not realisable (no operational action corresponds to the description).

Remark

The distinction «realisable / non-realisable» is a property of the formal term as a pair (description, register). It only tells whether the formal term has an ontological correlate. The

formal term itself does not become «better» or «worse» from the presence or absence of realisation; in the formal register all terms are equal.

The distinction matters only when we wish to **translate** results of the formal register into the ontological — this is the subject of Part IV.

II.8. Connection with proof theory

The two-register system of VR-Forms structurally resembles several constructions known in logic.

Two-sorted logic. A standard extension of first-order logic in which variables are divided into sorts. VR-Forms is formally a two-sorted system, with sorts «operational set» and «formal term». However, the sorts in VR-Forms are not symmetric: the formal sort **includes** the operational (every operational set has a formal correlate).

Conservative extensions. An extension of a theory is called conservative if no new statements about old objects are provable in it. Part III will show: VR-Forms is conservative over VR-Sets in the ontological register. The formal register does not allow new statements about operational sets to be proved; it only provides a language for talking about the non-operational.

Realisability (Kleene, Kreisel). A notion linking constructive mathematics to classical: a formula is considered «realisable» if it has a constructive witness. The notion of operational realisability in §II.7 is an analogue of realisability specialised to the VR-ontology.

Internal set theory (Nelson). An extension of ZFC separating standard and nonstandard objects. Structurally similar, but the motivation is opposite: IST introduces the nonstandard as a tool for proofs about the standard. VR-Forms introduces the formal as a way to speak about the non-existent without claiming proofs.

A detailed comparison with these systems — Part VII.

II.9. Summary of Part II

Established:

- (1) Alphabet and syntax of VR-Forms coincide with VR-Sets, with the addition of the separator $\ulcorner \cdot \urcorner$ for indicating the register (§II.1, §II.2).
- (2) A formal term is a pair (syntactic record, formal register); notation $\ulcorner \tau \urcorner$ (Definition II.2, §II.3).
- (3) The principle of forms: every syntactically correct record is a formal term. Unconditional, in contrast with the conditional closure principle (§II.4).
- (4) Logic and operations are classical, applied to formal terms in the usual way (§II.5).
- (5) Systematic comparison of formal term and operational set (§II.6).
- (6) The notion of operationally realisable formal term (Definition II.3, §II.7).

(7) Connections with two-sorted logic, conservative extensions, realisability, IST (§II.8).

What follows. Part III proves consistency of the extension: VR-Forms is conservative over VR-Sets in the ontological register. This theorem secures the safety of work in the formal register — no new ontological commitments are generated.